



Guide 7: User Profiles & Personalization — Complete PRD

"Har User Ke Liye Alag Experience"

Document:	User Profiles & Personalization — Feature PRD v1.0
Feature:	Cross-Cutting — All Modes Use This
Date:	June 2026
Status:	Final — Ready for Development

1. Feature Overview

User Profiles & Personalization system har AI agent ke output ko user ki specific constraints ke hisaab se adjust karta hai. Ye static template nahi — dynamic context injection hai. Same problem, different user = completely different ideas, blueprints, aur tech stacks.

Key Principle: Personalization is not 'filtering' — it's 'adapting'. AI doesn't just remove options; it reimagines the solution for that specific user.

2. User Profile Schema

```
model User { id String @id @default(uuid()) email String @unique name String? //
Core constraints skillLevel String // BEGINNER | INTERMEDIATE | ADVANCED budget Int
// in USD, 0 = free only teamSize Int // 1 = solo, 2-3 = small, 5+ = large timeline
String // e.g., "2 weeks", "1 month", "3 months" // Tech preferences techPrefs
String[] // ["React", "Node", "Python", "Flutter"] // Learning & goals
learningGoals String[] // ["Full-stack", "AI/ML", "Mobile"] industry String? //
"Education", "Healthcare", "FinTech" // Experience yearsOfExperience Int? // 0-2,
3-5, 5+ pastProjects Project[] // Platform settings preferredPlatform String? //
WEB, MOBILE, AI_ML, etc. stylePreference String? // MODERN, MINIMAL, CYBERPUNK
createdAt DateTime @default(now()) updatedAt DateTime @updatedAt }
```

3. Personalization Rules by Attribute

Skill Level Impact

BEGINNER: Simple HTML/CSS/JS, no frameworks, client-side where possible, free hosting (GitHub Pages), no AI APIs (use keyword matching), copy-paste ready code. INTERMEDIATE: Frameworks allowed (React/Vue), basic backend (Express/FastAPI), simple database (SQLite/MongoDB), basic OpenAI integration. ADVANCED: Microservices, event-driven architecture, vector databases, custom AI models, CI/CD, cloud-native (AWS/GCP).

Budget Impact

\$0: GitHub Pages, Supabase free tier, Vercel hobby, no paid APIs, open-source only, no third-party services. \$100-500: Basic cloud (DigitalOcean), essential paid APIs (OpenAI basic), shared database, basic monitoring. \$500+: AWS/GCP/Azure, managed services, premium APIs, dedicated servers, advanced analytics, professional tools.

Team Size Impact

1 (Solo): Monolithic architecture, single codebase, minimal coordination, simple git workflow, no CI/CD needed. 2-3 (Small): Simple collaboration, basic branching strategy (Git Flow), shared database, basic documentation. 5+ (Large): Microservices, full CI/CD, code review process, project management tools, comprehensive documentation, API contracts.

Timeline Impact

2 weeks: MVP only, core features, no polish, no tests, manual deployment, basic auth. 1 month: MVP + 2-3 enhancements, basic tests, simple CI/CD, responsive design. 3+ months: Full feature set, comprehensive testing, advanced CI/CD, optimization, monitoring, documentation.

Tech Preferences Impact

User ke preferred technologies ko prioritize karta hai. Agar user ne 'React' aur 'Node' select kiya hai, toh system Python/Django suggest nahi karega. Agar user 'Flutter' janta hai, toh mobile app ideas Flutter ke saath generate honge.

4. Context Injection into AI Agents

Har AI agent ke system prompt mein user profile context inject hota hai. Ye 'one-size-fits-all' approach ko reject karta hai. Same agent, different user = different output.

```
// Context Injection Function function buildPersonalizationContext(userProfile) {
return `USER PROFILE CONTEXT: - Skill Level: ${userProfile.skillLevel} - Budget:
${userProfile.budget} - Team Size: ${userProfile.teamSize} person(s) - Timeline:
${userProfile.timeline} - Tech Preferences: ${userProfile.techPrefs.join(', ')} -
Industry: ${userProfile.industry || 'General'} - Experience:
${userProfile.yearsOfExperience || 0} years PERSONALIZATION RULES: 1. Suggest ONLY
technologies user knows or wants to learn 2. Complexity must match skill level and
timeline 3. Budget constraints strictly enforced – no paid services for $0 budget
4. Team size determines architecture (monolith vs microservices) 5. Industry
context should influence feature suggestions 6. If user is beginner, explain WHY
each tech choice is made 7. If user is advanced, focus on architecture decisions and
trade-offs `; } // Usage in every agent const generatorPrompt = `
${buildPersonalizationContext(user)} PROBLEM: ${userInput} Generate 5-8 ideas
following the above constraints. `;
```

5. Example: Same Problem, Different Outputs

Problem: Students struggle to find internships

User A: Beginner, \$0, Solo, 2 weeks, HTML/CSS/JS

Ideas: Simple Resume Checker (client-side keyword matching), Internship Tracker Spreadsheet (Google Sheets API), College Job Board (static HTML). Tech: HTML + CSS + Vanilla JS, GitHub Pages, no backend. Blueprint: 1-page website, no database, localStorage for data.

User B: Intermediate, \$100, 2 people, 1 month, React/Node

Ideas: AI Resume Analyzer (OpenAI API), Internship Aggregator (web scraping), Skill Gap Analyzer. Tech: React + Node + MongoDB, Vercel + Render, OpenAI basic tier. Blueprint: Full-stack app, JWT auth, MongoDB, REST API.

User C: Advanced, \$5000, 5 people, 3 months, Next.js/Python/AI

Ideas: AI Career Coach (GPT-4 + vector search), Predictive Internship Matching (ML model), Multi-university Platform. Tech: Next.js + tRPC + PostgreSQL + Pinecone + Python FastAPI + AWS. Blueprint: Microservices, event-driven, vector search, CI/CD, monitoring.

6. Onboarding Flow

New user registration ke baad quick onboarding survey hota hai. 5-6 questions, 2 minutes max. Profile automatically AI agents mein inject ho jata hai.

- Q1: What is your current skill level? (Beginner / Intermediate / Advanced)
- Q2: What is your budget for this project? (\$0 / \$100-500 / \$500+)
- Q3: How many people in your team? (Just me / 2-3 / 5+)
- Q4: How much time do you have? (2 weeks / 1 month / 3+ months)
- Q5: Which technologies do you know or want to use? (Multi-select: React, Node, Python, Flutter, etc.)
- Q6: What industry are you targeting? (Education, Healthcare, FinTech, etc. — optional)

7. Interview Talking Points

Why Not Static Templates?

Static templates mein 'beginner web app', 'intermediate web app', 'advanced web app' hote hain. Ye rigid hai. Dynamic personalization mein same idea har user ke liye reimagined hota hai. Tech stack, complexity, features — sab adapt hote hain. Ye 'adaptive intelligence' hai.

How Do You Handle Conflicting Constraints?

Example: User beginner hai, par 3-month timeline hai. System intermediate-level ideas suggest karega with learning path. Ya user \$0 budget, par AI feature chahiye — system free alternatives batayega (Hugging Face, open-source models). Conflict resolution: prioritize feasibility, suggest alternatives.

What About Learning Goals?

Agar user 'AI/ML' seekhna chahata hai, toh system AI-heavy ideas suggest karega even if simpler alternatives exist. Learning goal ko priority deta hai over pure feasibility. Ye 'growth-oriented' personalization hai.

How Does Past Project History Help?

Vector search se similar past ideas find karta hai. System batata hai: 'Ye idea tumhare past project X se 70% similar hai. Consider evolving X instead.' Ya: 'Tumne pehle React use kiya, isliye ye idea React mein suggest kar rahe hain.'

— End of Guide 7: User Profiles & Personalization PRD —